



Министерство науки и высшего образования Российской Федерации  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
«Московский государственный технический университет  
имени Н. Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н. Э. Баумана)

---

ФАКУЛЬТЕТ «Информатика и системы управления»

---

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

---

## ОТЧЕТ

по лабораторной работе № 6, 7, 8

по курсу «Защита информации»

на тему: «Разработка алгоритма симметричного шифрования (AES)»

Студент ИУ7-71Б  
(Группа)

\_\_\_\_\_  
(Подпись, дата)

Я. Р. Овчинников  
(И. О. Фамилия)

Преподаватель

\_\_\_\_\_  
(Подпись, дата)

Ю. С. Руденкова  
(И. О. Фамилия)

2025 г.

## СОДЕРЖАНИЕ

|                               |    |
|-------------------------------|----|
| ЗАДАНИЕ                       | 3  |
| ТЕОРЕТИЧЕСКАЯ ЧАСТЬ           | 4  |
| РЕАЛИЗАЦИЯ                    | 14 |
| ДЕМОНСТРАЦИЯ РАБОТЫ ПРОГРАММЫ | 18 |

# ЗАДАНИЕ

## Цель работы

Разработка алгоритма шифрования и расшифрования произвольного файла текстового файла и архивного файла с помощью алгоритма симметричного шифрования AES.

## Задание

Реализовать в виде программы алгоритм AES-256, обеспечить шифрование и дешифрование произвольного файла текстового файла и архивного файла. Предусмотреть работу программы с пустым, однобайтовым файлом.

# ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

## Вопросы из лабораторной 5

### 1. Виды симметричного шифрования (поточные и блочные). Схема для одного из видов

Симметричное шифрование — это метод криптографии, в котором для шифрования и расшифровки используется один и тот же секретный ключ. Существуют два основных вида симметричного шифрования:

#### 1.1. Поточное (потокосное) шифрование

**Поточное шифрование** обрабатывает данные побитово или побайтово. Генератор создаёт поток ключевой последовательности, которая складывается с исходным текстом операцией XOR:

$$C_i = M_i \oplus K_i$$

где  $C_i$  — шифротекст,  $M_i$  — открытый текст,  $K_i$  — ключевой поток.

**Примеры поточных шифров:** RC4, ChaCha20, Salsa20, A5/1.

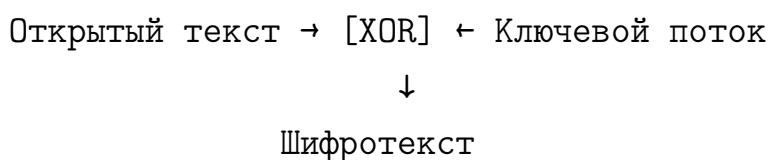
**Преимущества:**

- Высокая скорость работы
- Низкие требования к памяти
- Подходит для потоковых данных (аудио, видео)

**Недостатки:**

- Критична синхронизация потока
- Повторное использование ключа крайне опасно

**Схема поточного шифрования:**





## Структура алгоритма AES

Алгоритм AES состоит из следующих этапов:

- 1) **Key Expansion (Расширение ключа):** Из исходного ключа генерируются раундовые ключи для каждого раунда шифрования. Для AES-256 генерируется 15 раундовых ключей (60 32-битных слов).
- 2) **Initial Round (Начальный раунд):** Применяется операция `AddRoundKey` — XOR блока данных с первым раундовым ключом.
- 3) **Main Rounds (Основные раунды):** Для AES-256 выполняется 13 основных раундов (раунды 1-13). Каждый раунд включает четыре операции:
  - **SubBytes:** Нелинейная подстановка байтов через S-блок
  - **ShiftRows:** Циклический сдвиг строк матрицы состояния
  - **MixColumns:** Перемешивание столбцов матрицы состояния
  - **AddRoundKey:** XOR с раундовым ключом
- 4) **Final Round (Финальный раунд):** 14-й раунд выполняется без операции `MixColumns`:
  - `SubBytes`
  - `ShiftRows`
  - `AddRoundKey`

## Расшифровка AES

Расшифровка выполняется тем же алгоритмом с использованием обратных операций (`InvSubBytes`, `InvShiftRows`, `InvMixColumns`) и раундовых ключей в обратном порядке.

## Генерация раундовых ключей (Key Expansion)

Для AES-256:

- Исходный 256-битный ключ делится на 8 32-битных слов

- Всего генерируется 60 32-битных слов (4 слова  $\times$  15 раундов)
- Используются операции RotWord (циклический сдвиг), SubWord (подстановка через S-блок) и XOR с константами Rcon

**3. Определения алгоритмов перестановки и подстановки. Примеры каждого из этих видов алгоритмов. Пример алгоритма, использующего оба подхода**

### **3.1. Перестановка (Permutation)**

**Перестановка** — это криптографическое преобразование, которое переставляет биты или байты входной последовательности в определённом порядке согласно таблице перестановок.

Для входа  $X = x_1x_2\dots x_n$  и таблицы перестановок  $P = [p_1, p_2, \dots, p_m]$  выход определяется как:

$$y_i = x_{p_i}, \quad i = 1, 2, \dots, m$$

**Свойства перестановки:**

- Линейность
- Обратимость
- Обеспечивает диффузию (распространение влияния каждого бита)

**Примеры перестановок в криптографии:**

- Начальная перестановка (IP) в DES
- Расширение E в DES ( $32 \rightarrow 48$  бит)
- Перестановка P после S-блоков в DES
- ShiftRows в AES

## 3.2. Подстановка (Substitution)

**Подстановка** — это нелинейное преобразование, заменяющее входную последовательность на другую согласно таблице замен (S-box):

$$y = S(x)$$

### Свойства подстановки:

- Нелинейность
- Обеспечивает конфузию (скрывает связь между ключом и шифротекстом)
- Устойчивость к линейному и дифференциальному криптоанализу

### Примеры подстановок в криптографии:

- S-блоки в DES (8 таблиц, 6 бит  $\rightarrow$  4 бита)
- S-box в AES (таблица  $16 \times 16$ , 8 бит  $\rightarrow$  8 бит)
- Шифр Цезаря (простейшая подстановка)

### SubBytes в AES:

В AES операция SubBytes заменяет каждый байт матрицы состояния на соответствующее значение из S-блока. S-блок построен на основе обратного элемента в поле Галуа  $GF(2^8)$  с последующим аффинным преобразованием.

## 3.3. Алгоритм, использующий оба подхода

**AES** является классическим примером SP-сети (Substitution-Permutation Network) — алгоритма, комбинирующего подстановки и перестановки:

### Подстановки в AES:

- SubBytes — нелинейная подстановка через S-блок, обеспечивает конфузию

### Перестановки в AES:

- ShiftRows — циклический сдвиг строк матрицы состояния

- MixColumns — линейное преобразование столбцов (можно рассматривать как обобщённую перестановку)
- Key Expansion использует перестановки (RotWord)

### **Принцип работы:**

Чередование операций подстановки и перестановки в 14 раундах (для AES-256) создаёт высокую криптографическую стойкость:

- **Перестановки** обеспечивают **диффузию** — распространение влияния каждого бита входа на весь блок выхода
- **Подстановки** обеспечивают **конфузию** — усложнение статистической связи между ключом и шифротекстом

Благодаря многократному применению этих операций, изменение даже одного бита входных данных приводит к изменению в среднем половины бит выходных данных (лавинный эффект).

## **Вопросы из лабораторной 6**

### **1. Классификация автоматизированных систем с точки зрения требований безопасности (3 класса защищённости)**

Автоматизированные системы классифицируются по требованиям безопасности на основе важности обрабатываемой информации и потенциальных угроз:

#### **Класс 3 (низкий уровень защищённости):**

- Обработывается информация ограниченного распространения
- Базовые требования: аутентификация пользователей, журналирование событий, антивирусная защита
- Пример: корпоративные информационные системы без критичных данных

#### **Класс 2 (средний уровень защищённости):**

- Обрабатывается конфиденциальная информация
- Дополнительные требования: контроль доступа, шифрование данных, резервное копирование, защита от несанкционированного доступа
- Пример: банковские системы, медицинские информационные системы

### **Класс 1 (высокий уровень защищённости):**

- Обрабатывается информация, составляющая государственную тайну
- Строгие требования: многоуровневая аутентификация, криптографическая защита всех каналов, физическая защита оборудования, сертифицированные средства защиты
- Пример: государственные информационные системы специального назначения

## **2. Виды симметричного шифрования (поточные и блочные)**

Данный вопрос был подробно рассмотрен в Блоке 1, пункт 1.

## **3. Особенности алгоритма шифрования архивных файлов**

При шифровании архивных файлов необходимо учитывать следующие особенности:

### **3.1. Структура архивных файлов**

Архивные файлы (ZIP, RAR, 7z) имеют внутреннюю структуру:

- Заголовки с метаданными
- Сжатое содержимое файлов
- Каталог архива

## 3.2. Подходы к шифрованию архивов

### Шифрование на уровне архиватора:

- Многие архиваторы поддерживают встроенное шифрование (например, ZIP с AES)
- Шифруется только содержимое файлов, метаданные остаются открытыми
- Преимущество: возможность просмотра списка файлов без расшифровки

### Шифрование архива целиком (внешнее шифрование):

- Архив рассматривается как обычный бинарный файл
- Шифруется весь файл, включая заголовки и метаданные
- Преимущество: максимальная конфиденциальность (невозможно определить даже структуру архива)
- Недостаток: для доступа к любому файлу требуется полная расшифровка

## 3.3. Особенности реализации

В данной работе используется подход внешнего шифрования:

- Архивный файл обрабатывается как последовательность байт
- Применяется блочное шифрование AES-256 в режиме ECB
- Zero padding для последнего блока
- Сохранение исходного размера файла для корректной расшифровки

Преимущества такого подхода:

- Универсальность (подходит для любых файлов)
- Простота реализации
- Полная конфиденциальность (невозможно определить тип файла)

## **Вопросы из лабораторной 7**

### **1. Перечислите подсистемы защиты информации в автоматизированных системах**

Основные подсистемы защиты информации в автоматизированных системах:

#### **1.1. Подсистема идентификации и аутентификации**

- Проверка подлинности пользователей и процессов
- Методы: пароли, биометрия, электронные ключи, двухфакторная аутентификация

#### **1.2. Подсистема управления доступом**

- Контроль прав доступа к ресурсам системы
- Модели: дискреционная (DAC), мандатная (MAC), ролевая (RBAC)

#### **1.3. Подсистема шифрования данных**

- Защита информации при хранении и передаче
- Симметричное и асимметричное шифрование, криптографические протоколы

#### **1.4. Подсистема регистрации и аудита**

- Журналирование событий безопасности
- Анализ действий пользователей и системных процессов

#### **1.5. Подсистема антивирусной защиты**

- Обнаружение и нейтрализация вредоносного ПО
- Регулярное обновление антивирусных баз

## **1.6. Подсистема резервного копирования**

- Создание копий критичных данных
- Обеспечение восстановления после сбоев или атак

## **1.7. Подсистема межсетевого экранирования**

- Фильтрация сетевого трафика
- Защита от внешних и внутренних угроз

## **2. Описание алгоритма шифрования AES**

Данный вопрос был подробно рассмотрен в Блоке 1, пункт 2.

## **3. Особенности алгоритма симметричного шифрования архивных файлов**

Данный вопрос был подробно рассмотрен в Блоке 2, пункт 3.

## РЕАЛИЗАЦИЯ

На листинге 1 приведена реализация функции генерации раундовых ключей для AES-256.

Листинг 1 – Генерация раундовых ключей

```
1 void AES::key_expansion() {
2     int nk = 8; // Количество 32-битных слов в ключе для AES-256
3     int nr = 14; // Количество раундов для AES-256
4
5     // Копируем исходный ключ в первые nk слов
6     for (int i = 0; i < nk; ++i) {
7         round_keys[i] = (key[4*i] << 24) | (key[4*i+1] << 16) |
8             (key[4*i+2] << 8) | key[4*i+3];
9     }
10
11     // Генерируем остальные раундовые ключи
12     for (int i = nk; i < 4 * (nr + 1); ++i) {
13         uint32_t temp = round_keys[i - 1];
14
15         if (i % nk == 0) {
16             temp = sub_word(rot_word(temp)) ^
17                 (AESTable::RCON[i/nk - 1] << 24);
18         } else if (i % nk == 4) {
19             temp = sub_word(temp);
20         }
21         round_keys[i] = round_keys[i - nk] ^ temp;
22     }
23 }
```

Функция `key_expansion` генерирует 60 32-битных слов раундовых ключей из исходного 256-битного ключа. Используются операции `RotWord` (циклический сдвиг), `SubWord` (подстановка через S-блок) и XOR с константами раунда `Rcon`. На листинге 2 приведена реализация функции шифрования одного 128-битного блока.

Листинг 2 – Шифрование блока

```
1 void AES::encrypt_block(const uint8_t* input, uint8_t* output) {
2     uint8_t state[4][4];
```

```

3     bytes_to_state(input, state);
4
5     // Initial round
6     add_round_key(state, 0);
7
8     // Main rounds (1-13)
9     for (int round = 1; round < 14; ++round) {
10         sub_bytes(state);
11         shift_rows(state);
12         mix_columns(state);
13         add_round_key(state, round);
14     }
15
16     // Final round (без MixColumns)
17     sub_bytes(state);
18     shift_rows(state);
19     add_round_key(state, 14);
20
21     state_to_bytes(state, output);
22 }

```

Функция `encrypt_block` реализует основной алгоритм шифрования AES-256. Входной 16-байтовый блок преобразуется в матрицу состояния  $4 \times 4$ , затем последовательно применяются раундовые преобразования. На листинге 3 приведена реализация основных операций AES: `SubBytes` и `MixColumns`.

### Листинг 3 – Операции `SubBytes` и `MixColumns`

```

1 void AES::sub_bytes(uint8_t state[4][4]) {
2     for (int i = 0; i < 4; ++i) {
3         for (int j = 0; j < 4; ++j) {
4             state[i][j] = AESTable::S_BOX[state[i][j]];
5         }
6     }
7 }
8
9 void AES::mix_columns(uint8_t state[4][4]) {
10    for (int i = 0; i < 4; ++i) {
11        uint8_t a[4];
12        for (int j = 0; j < 4; ++j) {
13            a[j] = state[j][i];
14        }

```

```

15
16     state[0][i] = gmul(0x02, a[0]) ^ gmul(0x03, a[1]) ^ a[2]
17         ^ a[3];
18     state[1][i] = a[0] ^ gmul(0x02, a[1]) ^ gmul(0x03, a[2])
19         ^ a[3];
20     state[2][i] = a[0] ^ a[1] ^ gmul(0x02, a[2]) ^
21         gmul(0x03, a[3]);
22     state[3][i] = gmul(0x03, a[0]) ^ a[1] ^ a[2] ^
23         gmul(0x02, a[3]);
24 }
25 }

```

Операция SubBytes выполняет нелинейную подстановку каждого байта через S-блок. Операция MixColumns выполняет линейное преобразование столбцов матрицы состояния с использованием умножения в поле Галуа  $GF(2^{**} 8)$ . На листинге 4 приведена реализация функции шифрования файла с блочной обработкой данных.

#### Листинг 4 – Шифрование файла

```

1 void AES::encrypt_file(const std::string& input_path, const
2     std::string& output_path) {
3     if (!key_generated) {
4         throw DESExceptions::KeyNotGenerated();
5     }
6
7     std::ifstream input(input_path, std::ios::binary);
8     if (!input.is_open()) {
9         throw DESExceptions::FileReadError(input_path);
10    }
11
12    input.seekg(0, std::ios::end);
13    uint64_t file_size = input.tellg();
14    input.seekg(0, std::ios::beg);
15
16    std::ofstream output(output_path, std::ios::binary);
17    if (!output.is_open()) {
18        throw DESExceptions::FileWriteError(output_path);
19    }
20
21    // Записываем оригинальный размер файла
22    output.write(reinterpret_cast<const char*>(&file_size),

```

```

22         sizeof(file_size));
23     uint8_t buffer[16];
24     uint8_t encrypted[16];
25
26     while (true) {
27         input.read(reinterpret_cast<char*>(buffer), 16);
28         size_t bytes_read = input.gcount();
29
30         if (bytes_read == 0) break;
31
32         // Zero padding для последнего блока
33         if (bytes_read < 16) {
34             std::memset(buffer + bytes_read, 0, 16 - bytes_read);
35         }
36
37         encrypt_block(buffer, encrypted);
38         output.write(reinterpret_cast<const char*>(encrypted),
39                     16);
40     }
41     input.close();
42     output.close();
43 }

```

Функция `encrypt_file` выполняет шифрование произвольного файла. Файл читается блоками по 16 байт, каждый блок шифруется отдельно. В начало выходного файла записывается исходный размер для корректной расшифровки. Последний блок дополняется нулями до полного размера (zero padding).

# ДЕМОНСТРАЦИЯ РАБОТЫ ПРОГРАММЫ

На листинге ниже продемонстрирован основной сценарий работы программы:

Листинг 5 – Основной сценарий работы программы

```
1  lab6_7_8 git:(main) ./aes_cipher
2  === AES-256 Шифрование - Лабораторная работа ===
3
4  === AES-256 Шифрование ===
5  1. Сгенерировать ключ
6  2. Загрузить ключ из файла
7  3. Сохранить ключ в файл
8  4. Зашифровать файл
9  5. Расшифровать файл
10 6. Показать ключ
11 0. Выход
12 > 1
13
14 === Генерация ключа ===
15 Ключ успешно сгенерирован!
16
17 === AES-256 Шифрование ===
18 1. Сгенерировать ключ
19 2. Загрузить ключ из файла
20 3. Сохранить ключ в файл
21 4. Зашифровать файл
22 5. Расшифровать файл
23 6. Показать ключ
24 0. Выход
25 > 3
26
27 === Сохранение ключа ===
28 Введите имя файла для ключа (без расширения): first_key
29 Ключ сохранен: out/first_key.key
30
31 === AES-256 Шифрование ===
32 1. Сгенерировать ключ
33 2. Загрузить ключ из файла
34 3. Сохранить ключ в файл
35 4. Зашифровать файл
36 5. Расшифровать файл
```

```

37 6. Показать ключ
38 0. Выход
39 > 2
40
41 === Загрузка ключа ===
42 Введите имя файла ключа (без пути, в директории out/):
    first_key.key
43 Ключ успешно загружен
44
45 === AES-256 Шифрование ===
46 1. Сгенерировать ключ
47 2. Загрузить ключ из файла
48 3. Сохранить ключ в файл
49 4. Зашифровать файл
50 5. Расшифровать файл
51 6. Показать ключ
52 0. Выход
53 > 6
54
55 === AES Ключ ===
56 Ключ (hex):
    73e771cde1bc040c1367cfb52cb4f874862d99681b6b5b913e9722f74c7793e6
57
58 === AES-256 Шифрование ===
59 1. Сгенерировать ключ
60 2. Загрузить ключ из файла
61 3. Сохранить ключ в файл
62 4. Зашифровать файл
63 5. Расшифровать файл
64 6. Показать ключ
65 0. Выход
66 > 4
67
68 === Шифрование файла ===
69
70 Доступные файлы:
71 1. archive.zip
72 2. empty.txt
73 3. single.txt
74 4. test.txt
75

```

```

76 Выберите файл для шифрования (введите номер от 1 до 4, 0 для
    отмены): 1
77
78 Шифрование: archive.zip -> archive.zip.aes
79 Шифрование файла...
80 Размер файла: 245 байт
81 Обработано блоков: 16/16
82 Шифрование завершено успешно!
83 Зашифрованный файл сохранен: out/archive.zip.aes
84
85 === AES-256 Шифрование ===
86 1. Сгенерировать ключ
87 2. Загрузить ключ из файла
88 3. Сохранить ключ в файл
89 4. Зашифровать файл
90 5. Расшифровать файл
91 6. Показать ключ
92 0. Выход
93 > 5
94
95 === Расшифровка файла ===
96
97 Доступные файлы:
98 1. archive.zip.aes
99
100 Выберите файл для расшифровки (введите номер от 1 до 1, 0 для
    отмены): 1
101
102 Расшифровка: archive.zip.aes -> decrypted_archive.zip
103 Расшифровка файла...
104 Оригинальный размер: 245 байт
105
106 Расшифровка завершена успешно!
107 Расшифрованный файл сохранен: out/decrypted_archive.zip
108
109 === AES-256 Шифрование ===
110 1. Сгенерировать ключ
111 2. Загрузить ключ из файла
112 3. Сохранить ключ в файл
113 4. Зашифровать файл
114 5. Расшифровать файл

```

```

115 6. Показать ключ
116 0. Выход
117 > 0
118
119 Выход из программы.
120 lab6_7_8 git:(main) diff out/archive.zip
    out/decrypted_archive.zip
121 lab6_7_8 git:(main)

```

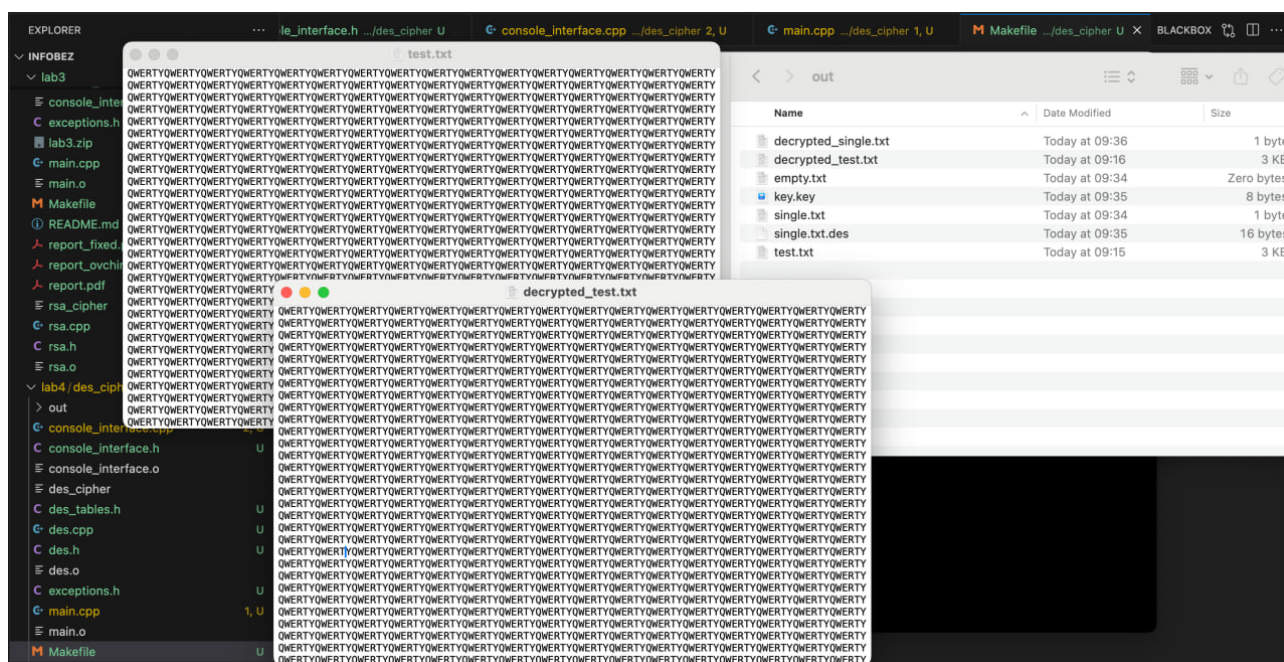


Рисунок 1 – Шифрование большого файла

Также была предусмотрена обработка большинства ошибочных ситуаций без экстренного завершения программы:

```
➤ des_cipher git:(main) x ./des_cipher
=== DES Шифрование - Лабораторная работа 2.2 ===

=== DES Шифрование ===
1. Сгенерировать ключ
2. Загрузить ключ из файла
3. Сохранить ключ в файл
4. Зашифровать файл
5. Расшифровать файл
6. Показать ключ
0. Выход
> 4
Ключ еще не сгенерирован. Сначала создайте или загрузите ключ.
```

Рисунок 2 – Обработка ошибочных ситуаций